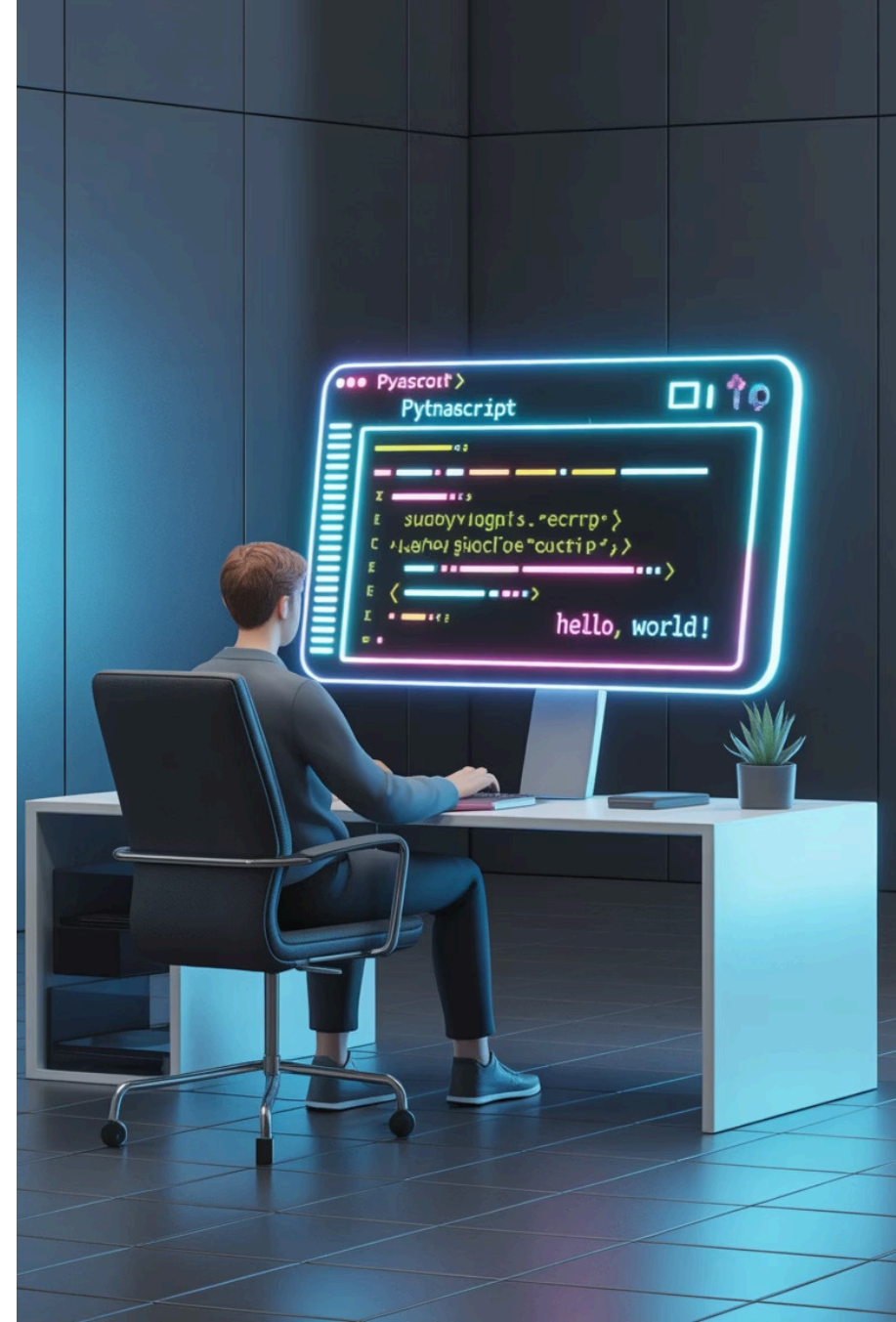


Introducción a las estructuras secuenciales en la programación

Una guía fundamental para programadores

by OscarLondero



Programa del curso

1

Fundamentos de programación

Conceptos básicos, variables y pensamiento computacional

2

Estructuras secuenciales

Comprensión de algoritmos, pseudocódigo y diagramas de flujo

3

Implementación en Python

Traducción de algoritmos en código Python

4

Ejercicios prácticos

Ejemplos prácticos y pruebas de escritorio

Objetivos del módulo

Comprender los fundamentos de la programación

Aprender qué es la programación y cómo funciona un programa

Desarrollar el pensamiento computacional

Comenzar a abordar los problemas desde una perspectiva computacional

Identificar la estructura algorítmica

Reconocer el patrón de **entrada-proceso-salida** en los algoritmos

Aplicar estructuras secuenciales

Usar pseudocódigo, diagramas de flujo y Python para resolver problemas

Practicar con pruebas de escritorio

Verificar la lógica mediante pruebas manuales de algoritmos



¿Qué es un programa?

Un **programa** es un conjunto ordenado de instrucciones que una computadora ejecuta para realizar una tarea específica.

Los programas son esencialmente recetas que les dicen a las computadoras exactamente qué hacer, paso a paso.

 Cada aplicación del teléfono, cada sitio web que se usa está ejecutando un programa. Siguiendo una serie de instrucciones.



¿Qué significa programar?

Programar significa escribir instrucciones utilizando un lenguaje que las computadoras puedan interpretar (como Python, Java, Php, C#, etc.).

Implica:

- Traducir ideas humanas en instrucciones informáticas
- Seguir una sintaxis y reglas específicas
- Crear soluciones a problemas a través del código



¿Qué es una variable?

Una **variable** es una **ubicación** de memoria con **nombre** donde se **almacenan** valores y pueden **cambiar** durante la ejecución del programa

Las variables pueden ser concebidas como contenedores etiquetados que contienen información.

```
# Ejemplo en Python  
name = "Ana"  
age = 20  
numero = 14
```

Las variables almacenan diferentes tipos de datos: números, texto e información más compleja.





Tipos de variables en Python

Enteros (int)

Números enteros sin decimales

```
age = 25  
count = -10
```

Punto flotante (float)

Números con decimales

```
price = 19.99  
temperature = -2.5
```

Cadenas (str)

Texto entre comillas

```
name = "Carlos"  
message = '¡Hola!'
```

Booleano (bool)

Valores verdadero o falso

```
is_student = True  
has_passed = False
```

Pensamiento computacional

El pensamiento computacional es un **enfoque de resolución de problemas** que nos ayuda a abordar problemas complejos de manera sistemática.

No sirve solamente para la programación, sino que es una habilidad valiosa para la vida en general.

Todo programador desarrolla una alta capacidad de resolución de problemas.



Cuatro pilares del pensamiento computacional



Descomposición

Dividir problemas complejos en partes más pequeñas y manejables. Por ejemplo, dividir un programa grande en funciones o módulos.



Reconocimiento de patrones

Identificar similitudes o repeticiones dentro o entre problemas. Esto ayuda a reutilizar soluciones y simplificar el código.



Abstracción

Centrarse solo en los detalles importantes e ignorar la información irrelevante. Esto ayuda a gestionar la complejidad en sistemas grandes.



Algoritmos

Crear procedimientos **paso a paso** para resolver problemas. Estas son las instrucciones precisas que forman nuestros programas.

¿Qué es una estructura secuencial?

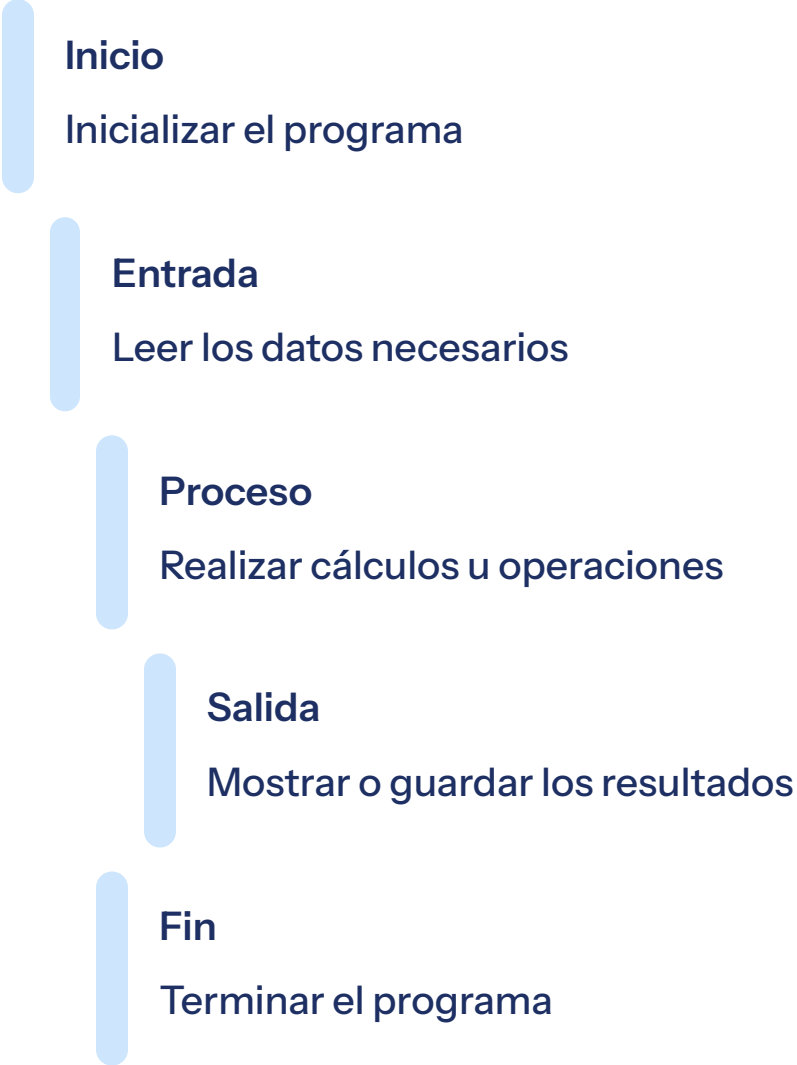
Una **estructura secuencial** es el patrón de programación más básico donde las instrucciones se ejecutan una tras otra, siguiendo un orden establecido.

La computadora lee y ejecuta cada línea de código exactamente una vez, de arriba a abajo.

- ❏ La ejecución secuencial es como seguir una receta: completas cada paso en orden antes de pasar al siguiente.



Modelo general de un algoritmo secuencial



Inicio

Inicializar el programa

Entrada

Leer los datos necesarios

Proceso

Realizar cálculos u operaciones

Salida

Mostrar o guardar los resultados

Fin

Terminar el programa

Estructura secuencial en Python

El código de Python sigue naturalmente una estructura secuencial, ejecutándose de arriba a abajo.

Ejemplo en python para aplicar un índice a un valor

Entrada

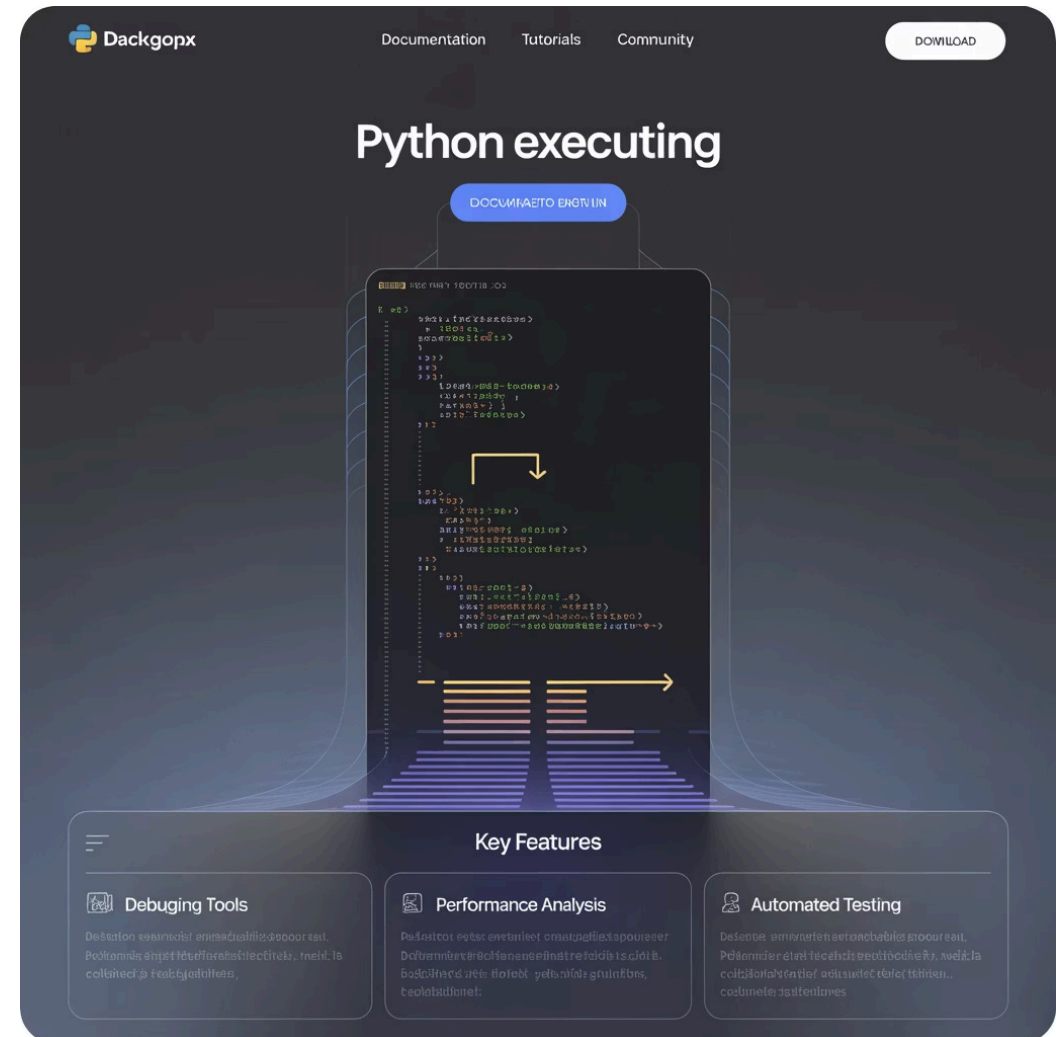
```
num = int(input("Ingrese un número: "))
```

Proceso

```
resultado = num * 2.12541
```

Salida

```
print("El valor final con índice es:", resultado)
```



Cada instrucción se ejecuta completamente antes de pasar a la siguiente. Esto crea un flujo de programa predecible.

Entrada en Python

La función **input()** lee datos del usuario:

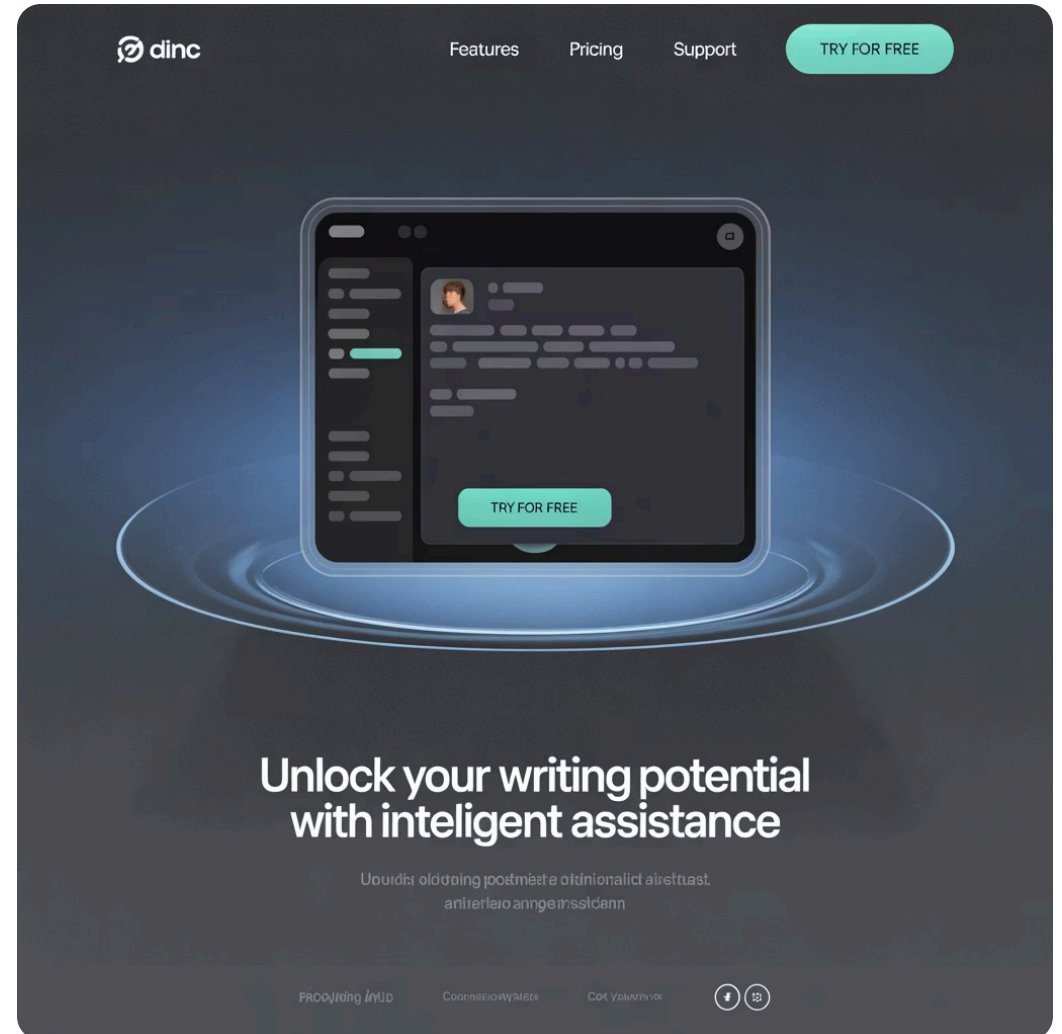
```
# Leyendo una cadena
name = input("Ingrese su nombre: ")

# Leyendo y convirtiendo a entero
age = int(input("Ingrese su edad: "))

# Leyendo y convirtiendo a flotante
height = float(input("Ingrese la altura en metros: "))
```

Al convertir la entrada al tipo de datos adecuado me aseguro de tratar los datos debidamente obteniendo los resultados esperados.

⚠ ¡La función **input()** de python siempre devuelve una cadena, por lo que debe ser convertirla a números cuando sea necesario!



Procesamiento en Python

Operaciones aritméticas

```
suma = num1 + num2  
diferencia = num1 - num2  
producto = num1 * num2  
cociente = num1 / num2
```

Operaciones de cadena

```
full_name = first_name + " " +  
last_name
```

Asignación de valor

```
total = precio * cantidad  
promedio = (x + y + z) / 3
```

Salida en Python

La función **print()** muestra información al usuario:

```
# Salida simple
print("¡Hola, mundo!")

# Salida con variables
name = "María"
print("Hola,", name)

# Salida formateada e interpolación
age = 25
print(f"Tu edad es {age} años")
```

La salida es la forma en que un programa comunica los resultados al usuario.

Una salida clara e informativa hace que los programas sean más fáciles de usar.



Ejemplo completo: Área de un rectángulo

Problema: Calcular el área de un rectángulo

Fórmula: $\text{área} = \text{largo} \times \text{ancho}$

```
# Cálculo del área de un rectángulo en Python

# ENTRADA
base = float(input("Ingrese el largo del rectángulo: "))
altura = float(input("Ingrese el ancho del rectángulo: "))

# PROCESO
área = base * altura

# SALIDA
print("El área del rectángulo es:", área)
```

Este programa sigue nuestra estructura secuencial: entrada (base, altura), proceso (cálculo) y salida (mostrar el área).



Pseudocódigo para el área de un rectángulo

El **pseudocódigo** es una descripción legible por humanos de un algoritmo que es más fácil de entender que el código real.

INICIO

ESCRIBIR "Ingrese la base del rectángulo:"

LEER base

ESCRIBIR "Ingrese la altura del rectángulo:"

LEER altura

$\text{area} \leftarrow \text{base} * \text{altura}$

ESCRIBIR "El área del rectángulo es:", area

FIN

El pseudocódigo ayuda a planificar su solución antes de escribir el código real, lo que hace que el proceso de programación sea más organizado.

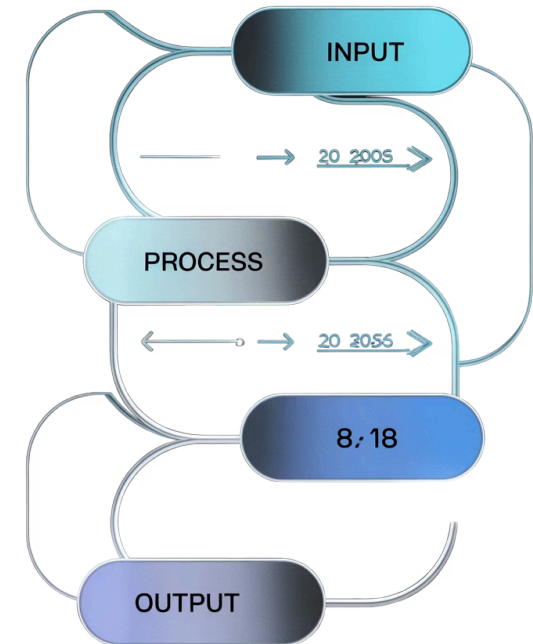


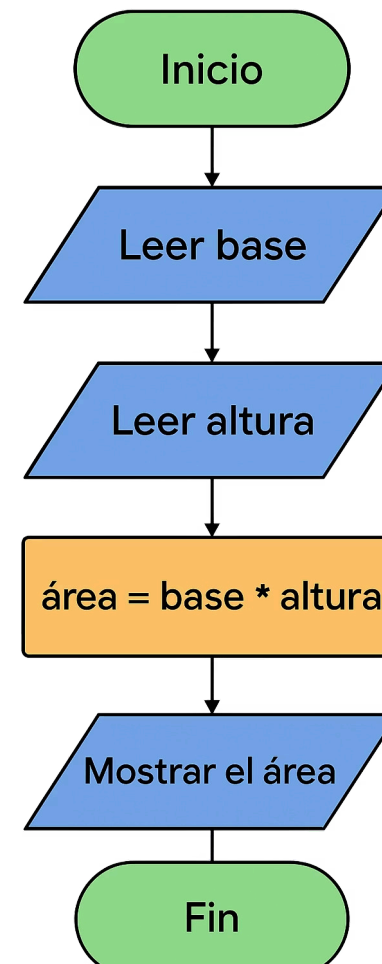
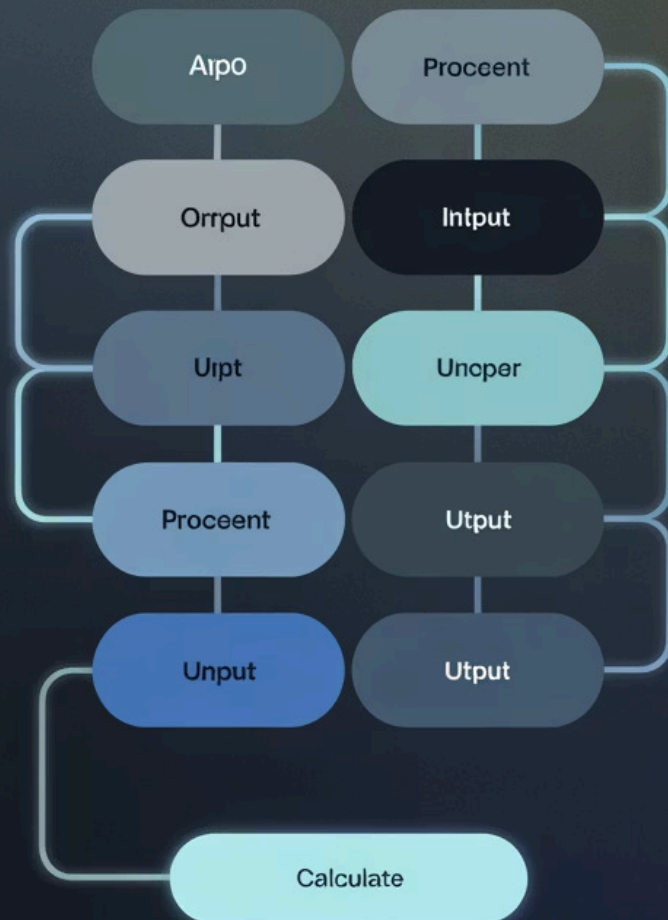
Diagramas de flujo

Un **diagrama de flujo** representa visualmente los pasos de un algoritmo utilizando símbolos estandarizados conectados por flechas.

- Óvalos: Inicio/Fin
- Paralelogramos: Entrada/Salida
- Rectángulos: Procesamiento
- Diamantes: Decisiones

Los diagramas de flujo facilitan la visualización del flujo del programa e identifican posibles problemas en su algoritmo.



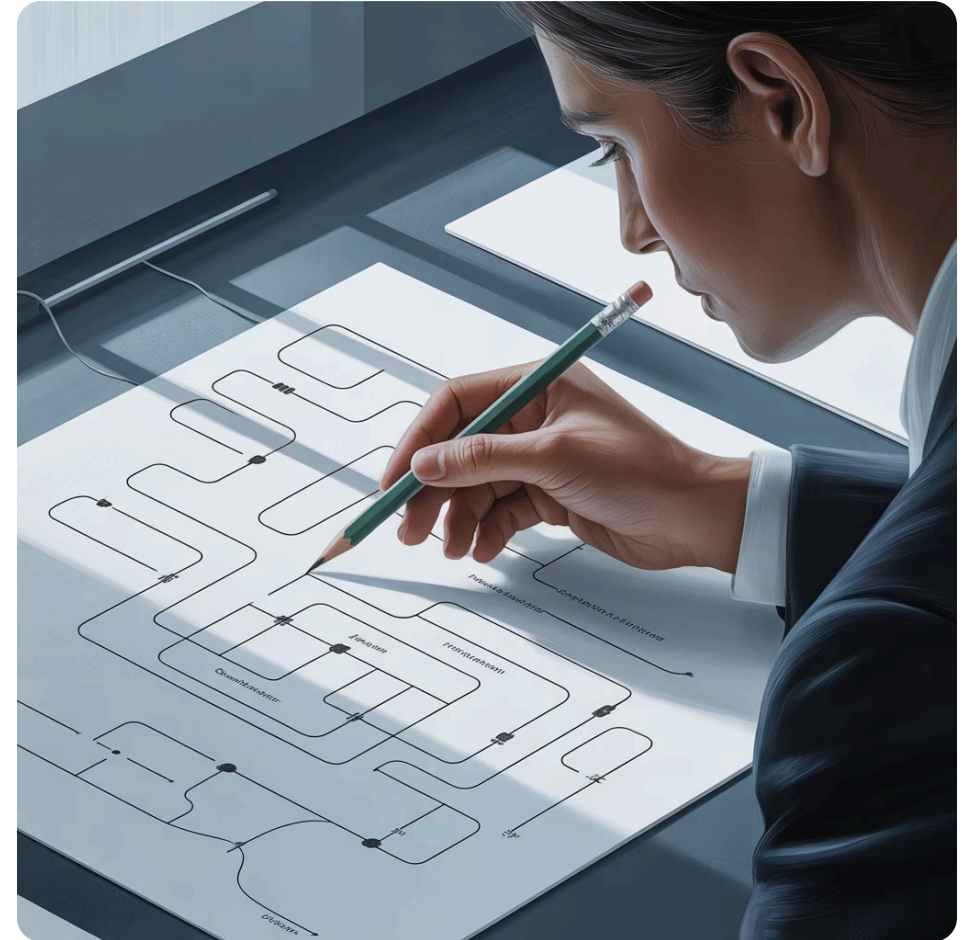


Pruebas de escritorio

Las **pruebas de escritorio** (o "simulación") consisten en rastrear manualmente un algoritmo con valores de entrada específicos para verificar su lógica.

Ayuda a detectar errores lógicos antes de escribir el código.

Las pruebas de escritorio realizan un seguimiento de cada paso o instrucción exactamente como está escrita para ver si el algoritmo produce los resultados esperados. Un Tema importantísimo para evitar errores lógicos.



Pruebas de escritorio: Área del rectángulo

Variable	Caso de prueba 1	Caso de prueba 2
base	5	7
altura	4	3
área = base × altura	$5 \times 4 = 20$	$7 \times 3 = 21$
Resultado	"El área del rectángulo es: 20"	"El área del rectángulo es: 21"

Al calcular manualmente los resultados esperados para diferentes entradas, podemos verificar la exactitud de nuestro algoritmo.

Ejercicio 1: Saludo personalizado

Problema: Crea un programa que pida el nombre de un usuario y muestre un saludo personalizado.

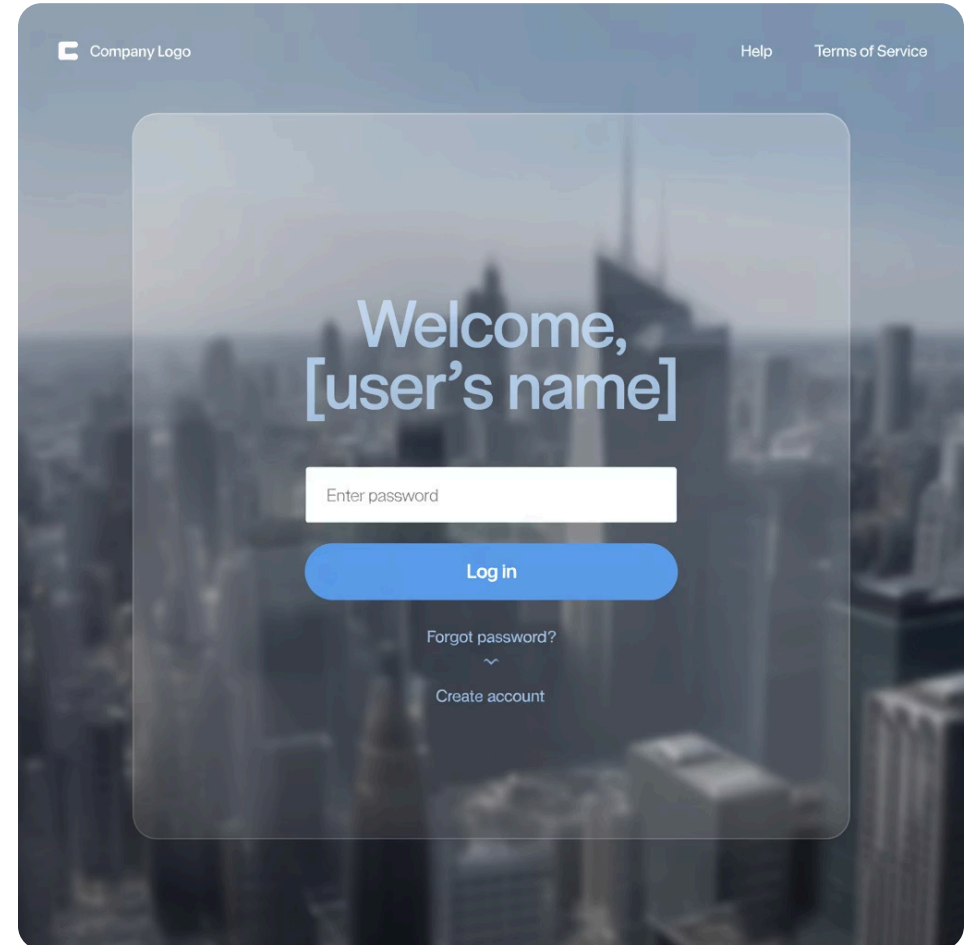
Pseudocódigo del algoritmo secuencial:

```
INICIO
  LEER nombre
  ESCRIBIR "Hola", nombre
FIN
```

Código de Python:

```
nombre = input("Ingresa tu nombre: ")
print("Hola,", nombre)
```

Este sencillo programa demuestra el patrón de entrada-salida de los programas secuenciales sin ningún paso de procesamiento.



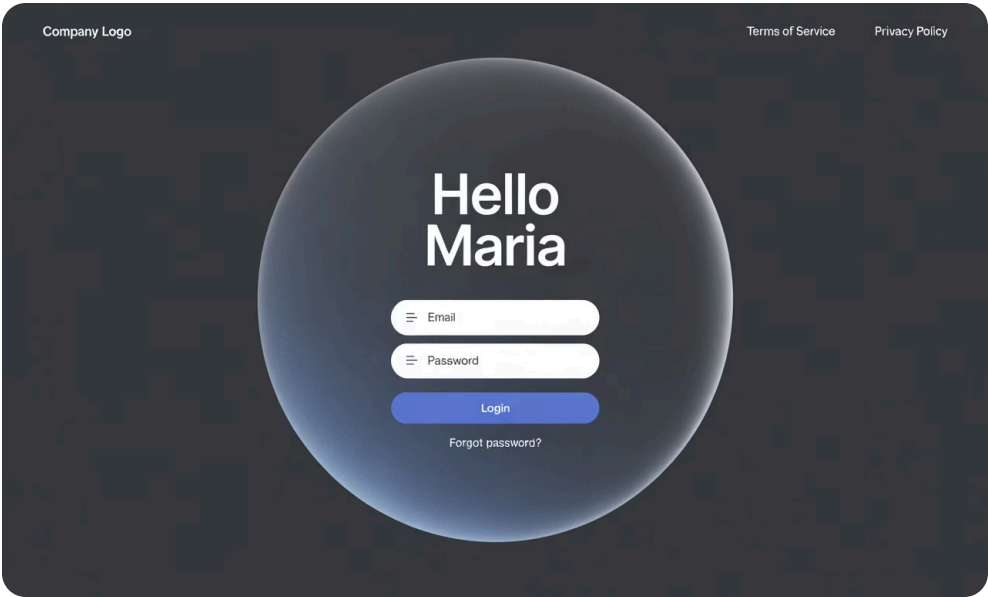
Ejercicio 1: Pruebas

1

Caso de prueba 1

Entrada: "María"

Salida esperada: "Hola, María"

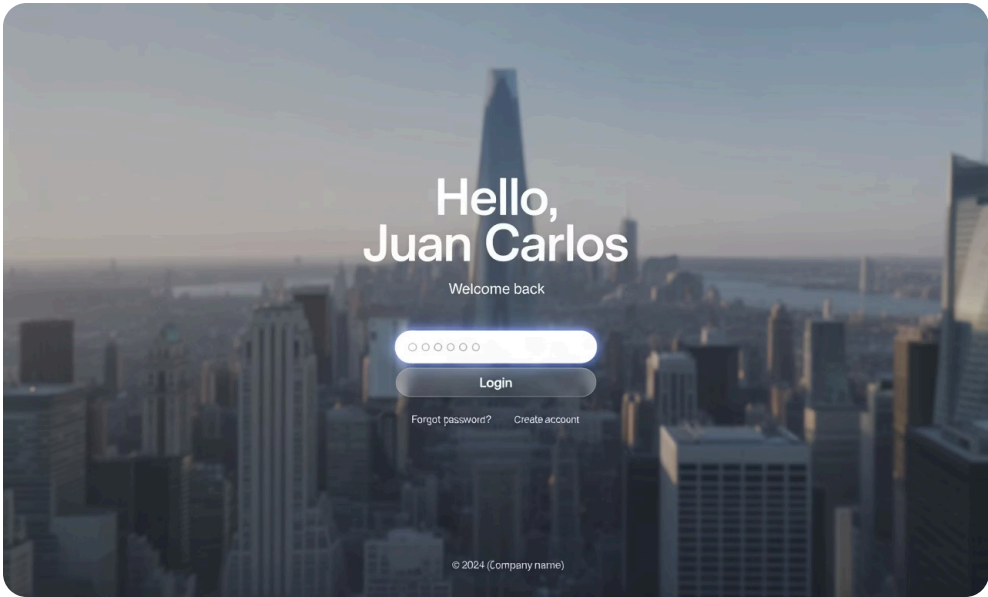


2

Caso de prueba 2

Entrada: "Juan Carlos"

Salida esperada: "Hola, Juan Carlos"



El programa maneja correctamente nombres de una y varias palabras.

Ejercicio 2: Sumar dos números

Problema: Crea un programa que sume dos números ingresados por el usuario.

Pseudocódigo:

```
INICIO  
  LEER num1, num2  
  suma ← num1 + num2  
  ESCRIBIR suma  
FIN
```

Código de Python:

```
num1 = int(input("Ingrese el primer número: "))  
num2 = int(input("Ingrese el segundo número: "))  
suma = num1 + num2  
print("La suma es:", suma)
```

Este programa muestra los tres pasos de la programación secuencial: entrada, proceso y salida.



Ejercicio 2: Pruebas

Variable	Caso de prueba 1	Caso de prueba 2
num1	5	-10
num2	7	15
sum = num1 + num2	5 + 7 = 12	-10 + 15 = 5
Resultado	"La suma es: 12"	"La suma es: 5"

Las pruebas de escritorio ayudan a verificar que nuestro programa funcione correctamente con números positivos y negativos.

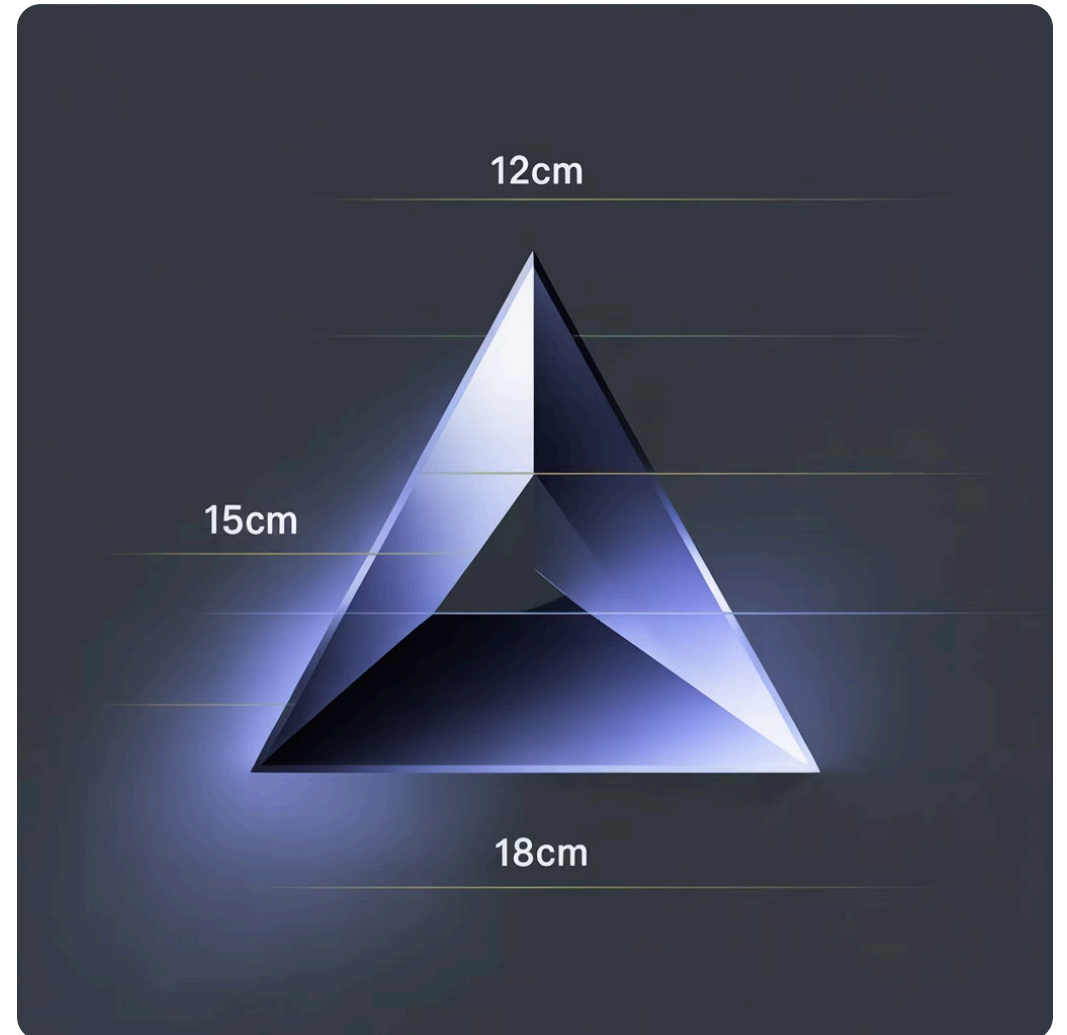
Ejercicio 3: Perímetro del triángulo

Problema: Calcula el perímetro de un triángulo dada la longitud de sus tres lados.

Código Python:

```
lado1 = float(input("Ingrese el lado 1: "))  
lado2 = float(input("Ingrese el lado 2: "))  
lado3 = float(input("Ingrese el lado 3: "))  
  
perimetro = lado1 + lado2 + lado3  
  
print("El perímetro es:", perimetro)
```

El perímetro de un triángulo es simplemente la suma de las longitudes de sus tres lados.



Ejercicio 3: Pruebas

1

Caso de prueba

Entrada:

- lado1 = 3
- lado2 = 4
- lado3 = 5

Cálculo: $\text{perímetro} = 3 + 4 + 5 = 12$

Salida: "El perímetro es: 12.0"

2

Caso de prueba

Entrada:

- lado1 = 5.5
- lado2 = 6.25
- lado3 = 7.75

Cálculo: $\text{perímetro} = 5.5 + 6.25 + 7.75 = 19.5$

Salida: "El perímetro es: 19.5"

Ejercicio 4: Descuento del producto

Problema: Calcula el precio final de un producto después de aplicar un porcentaje de descuento.

Código de Python:

```
price = float(input("Ingrese el precio del producto: "))
discount = float(input("Ingrese el descuento (%): "))

final_price = price - (price * discount / 100)

print("El precio con descuento es:", final_price)
```

Este programa aplica un descuento porcentual al precio de un producto, un cálculo común en aplicaciones de ventas y comercio electrónico.



Ejercicio 4: Pruebas

1

Caso de prueba 1

Entrada:

- precio = 100
- descuento = 20

Cálculo:

$\text{precio_final} = 100 - (100 \times 20 / 100) = 100 - 20 = 80$

Salida: "El precio con descuento es: 80.0"

2

Caso de prueba 2

Entrada:

- precio = 45.50
- descuento = 15

Cálculo:

$\text{precio_final} = 45.50 - (45.50 \times 15 / 100) = 45.50 - 6.825 = 38.675$

Salida: "El precio con descuento es: 38.675"



Conclusiones clave

- La estructura secuencial es la base**
Forma el bloque de construcción para todas las demás estructuras de programación.
- Patrón Entrada-Proceso-Salida**
La mayoría de los programas siguen esta estructura básica para resolver problemas.
- Múltiples representaciones**
El pseudocódigo, los diagramas de flujo y el código expresan el mismo algoritmo en diferentes formas.
- Las pruebas validan la corrección**
Las pruebas de escritorio ayudan a verificar su algoritmo antes de la implementación.
- La práctica desarrolla la habilidad**
La práctica hace al maestro!

A dark, semi-transparent background image showing a group of business professionals in a meeting. They are gathered around a large digital display that shows a complex flowchart or organizational chart. One man is pointing at the screen, while others look on attentively. The overall tone is professional and collaborative.

¡Gracias!

Esperamos que este módulo te haya proporcionado una base sólida en estructuras secuenciales de programación.

¡Continúa practicando!

Synergyos
Work smarter, together